

GETTING STARTED WITH HIGH PERFORMANCE COMPUTING IN RESEARCH

Richard Polzin <rpolzin@ukaachen.de>

LAST UPDATED: 22. APR 2026

TABLE OF CONTENT

1. Introduction

2. Linux Basics for HPC

- Command Line • Filesystem • Files • Text & Search
- Permissions • vim • Shell Scripts • Environment •
Config

3. SLURM Job Manager

4. Compute Time Application

5. Tips and Tricks

INTRODUCTION

OBJECTIVES

- Understand what HPC is
- Learn why HPC is essential for modern research
- Discover available HPC resources
- Walk through a project application process

WHAT IS HPC?

- **H**igh **P**erformance **C**omputing involves the use of supercomputers and parallel processing techniques to solve complex computational problems
- Combines computing resources for higher performance
- HPC Systems consist of clusters with interconnected nodes
- Enable processing larger datasets and complex simulations

WHAT IS HPC?

WHY IS HPC ESSENTIAL?

- **Speed:** Reduces time to results
- **Capacity:** Handle large scale data and simulation
- **Complexity:** Solve problems too complex for standard computers

HPC RESOURCES

- State of the art HPC facilities are available through the National High Performance Computing (NHR) network
- Including Compute Clusters, Large Scale Storage, and advanced networking infrastructure

ACCESS ELIGIBILITY AND REQUIREMENTS

- Researchers, students, and collaborators from German Universities
- University credentials (TIM-ID) are required
- Compliance with university and HPC usage policy is expected
- Resources are to be used responsibly

HPC RESOURCES

THE RWTH COMPUTE CLUSTER

Per Node:

- 2x Intel Xeon 8468 (48 core CPU)
- 1.5 TB local SSD Storage
- 256GB - 1,024GB of RAM
- 632 such nodes available

THE RWTH COMPUTE CLUSTER

Per Node (ML):

- 2x Intel Xeon 8468
- 695 GB local SSD Storage
- 512GB of RAM
- 4 x NVIDIA H100 GPU (96GB HBM2e)
- 52 such nodes available

ACCOUNT CREATION

Use Selfservice to create your account
(<https://idm.rwth-aachen.de/selfservice/>)

1. Accounts and Passwords
2. Account Overview
3. Create HPC Account

LOGIN

```
ssh ab1234@login23-1.hpc-itc.rwth-aachen.de
```

- Login with secure shell (ssh): **ssh username@host**
- Needs **VPN** if not connected to eduroam — see help.itc.rwth-aachen.de → "VPN"
- Different node types available for different use cases (see next slide)

CLUSTER ACCESS NODES

Use the appropriate node for your task.

Be considerate of shared resources.

- Login Nodes
 - lightweight tasks (script editing, testing)
- Copy Nodes
 - Optimized for data transfer

 overview of different login nodes

SSH CONFIGURATION

Typing the full hostname every time is tedious. Save presets in `~/.ssh/config`:

```
1 Host claix
2     HostName login18-1.hpc.itc.rwth-aachen.de
3     User ab123456
```

Then just use the alias:

```
1 ssh claix           # short alias – no hostname needed
2 ssh -X claix        # with X11 forwarding for graphical apps
```

MOUNTING THE CLUSTER FILESYSTEM

With `sshfs` you can browse cluster files in your local file manager:

```
1 sshfs ab123456@copy23node:/home/ab123456 /mnt/clusterhome  
2 sudo umount -l /mnt/clusterhome # unmount when done
```

Copy nodes (copy23node) are optimised for data transfer – always use them for large file moves.

SSH KEYS – SKIP THE PASSWORD

Use a public/private key pair instead of a password — more convenient, more secure.

```
1 # 1. Generate a key pair (on your local PC)
2 ssh-keygen -t ed25519
3 #     → id_ed25519 (private) and id_ed25519.pub (public) in ~
4
5 # 2. Copy public key to cluster
6 ssh-copy-id -i ~/.ssh/id_ed25519.pub claix
7 #     Windows: paste contents of .pub into ~/.ssh/authorized_
8
9 # 3. Log in – key used automatically
10 ssh claix
```

Use one key per device — if a laptop is stolen, remove just that key from ~/.ssh/authorized keys.

THE FILE SYSTEMS

- **\$HOME**: Small quota, backed up. For scripts and small files.
- **\$WORK**: Large quota, not backed up. For working with many small files.
- **\$HPCWORK**: Largest quota, not backed up. For I/O intense jobs and large files.

DATA TRANSFER METHODS

- Use copy nodes for data transfer
- Consider data security and encryption
- Terminal: *scp* or *rsync*
- Mount as folder: *sshfs*

PROJECT MANAGEMENT

- Groups are created for every project
- Every group consists of owners (PC/PI), Managers and Members
- Granted computation time and storage space is shared through groups
- In Aachen project storage is deleted **8 months** after a project's conclusion. Make sure you migrate the data by then!

PROJECT MANAGEMENT

Users can be added to and removed from groups/projects using their TIM-ID.

```
member add --name <project-id> <user-id>  
member delete --name <project-id> <user-id>  
member finger # view group affiliations
```

- append **--manager** to any command above to assign or revoke the manager role

THE COMMAND LINE

WHY LINUX?

Linux is everywhere — you just don't always see it:

Domain	Linux share
Supercomputers (Top 500)	100%
Web servers	~96%
Cloud infrastructure (AWS, Azure, GCP)	~90%
Android smartphones	~71% of mobile OS market
Desktop PCs	~4%

WHAT IS THE SHELL?

- A text interface where you type commands — also called **CLI**, terminal, or console
- **Advantages:** always available, scriptable, fast, works over SSH
- **Disadvantage:** no buttons — you need to know the commands

The shell is a program itself. On a cluster you're running a shell *inside* an SSH session — closing the session kills everything running in it.

*Default shell on most systems (including RWTH): **bash***

READING THE PROMPT

```
[js056352@login1 ~]$
```

Part	Meaning
js056352	Your username
login1	Hostname — which machine you're on
~	Current directory (~ = your home)
\$	Ready for input (# = you are root — be careful)

ANATOMY OF A COMMAND

```
1 $ hostname -f
2 login1.cm.cluster
3 $
```

- **Command** — what to do
- **Options** — short (`-t`) or long (`--time`): `sbatch -t 0:30:00` $\equiv$ `sbatch --time 0:30:00`
- **Output** — printed below; format varies by command
- New \$ prompt = done. **Case-sensitive** — `ls` $\neq$ `LS`.

ESSENTIAL SHORTCUTS

Key	Action
Tab	Auto-complete command or path
Tab Tab	List all possible completions
↑ / ↓	Navigate command history
Ctrl-C	Abort running command
Ctrl-Z	Suspend command (resume with fg)
Ctrl-D	Exit shell / send end-of-input

When stuck: **Ctrl-C** gets you back to a prompt.

GETTING HELP – AND STAYING SAFE

Finding the right command:

```
1 man <command>      # full manual page – press q to quit
2 <command> --help    # quick reference
```

The console has no Undo:

- rm deletes immediately — no recycle bin, no undo
- Never run a command you don't understand
- Always check your working directory before destructive commands
- Avoid running as root unless explicitly required

PROCESSES

A **process** is a running instance of a program — has a unique **PID**, inherits owner's permissions.

```
1 top          # live CPU/memory view — q to quit, u to filter by
2 kill <PID>   # terminate a process
```

Foreground and background:

```
command &      # start in background
# Ctrl-Z       # suspend foreground process
bg / fg        # resume in background / bring to foreground
jobs           # list background jobs
```

*On HPC, SLURM manages processes —
but you need this when something hangs*

DIRECTORY STRUCTURE

THE LINUX FILESYSTEM

No drive letters — everything lives under one tree rooted at /:

```
/
├── bin      ← System programs (ls, bash, ...)
├── etc      ← Configuration files
├── home     ← User home directories ← the only place you can write
│   └── ab123456/
├── tmp      ← Temporary files (cleared on reboot)
└── var      ← Logs and runtime data
```

- **Absolute path** starts with /:
/home/ab123456/data
- **Relative path** is relative to where you are: ./data
- pwd — print your current location
- Shortcuts: . (here) · .. (parent) · ~ (your home)

NAVIGATING

```
1 cd /work/ab123456    # change directory (absolute path)
2 cd ..
3 # go up one level
4 cd ~
5 # go to your home directory
6 cd -
7 # go back to previous directory
```

`pwd` prints your current location. which `<command>`
shows where a command lives.

LISTING FILES

```
1 ls          # list files
2 ls -l       # long format: permissions, size, date
3 ls -la      # include hidden files (names starting with .)
4 ls -lt      # sort by modification time
```

FILES

WORKING WITH FILES

Extensions don't matter to Linux — use `file <name>` to identify type. Key distinction: **text files** (editable, searchable) vs **binary files**.

Command	Action
<code>mv <old> <new></code>	Move or rename
<code>cp <src> <dest></code>	Copy (–r for directories)
<code>mkdir <name></code>	Create directory
<code>touch <name></code>	Create empty file / update

WILDCARDS (GLOBBING)

Select files by pattern — the shell expands them before the command runs:

Pattern	Matches
*	Any string (including empty)
?	Exactly one character
[abc]	One character from the set

```
rm *.log      # delete all .log files
ls report_?.txt # list report_1.txt, report_2.txt, ...
```

THE `find` COMMAND

```
1 find . -name "*.log" -type f
2 # find by name
3 find . -type f -mtime -7
4     # modified in last 7 days
5 find . -name "*.log" -exec rm {} \;      # act on results
```

*Always **quote** wildcard patterns passed to `find` – otherwise the shell expands them first before `find` sees them.*

TEXT, STREAMS, AND SEARCH

STREAMS AND REDIRECTION

Every process has three streams: **stdin** (input) · **stdout** (output) · **stderr** (errors).

```
1 command > file           # write stdout to file (overwrites)
2 command >> file          # append stdout to file
3 command 2> file          # write stderr to file
4 command > out 2> err     # stdout and stderr to separate files
```

Pipes — chain commands, passing stdout of one into stdin of the next:

```
cat results.txt | grep "ERROR" | sort | uniq -c
```

VIEWING FILES

```
1 cat <file>           # print whole file
2 less <file>          # scrollable viewer – q to quit, / to search
3 head -n 20 <file>    # first 20 lines
4 tail -f <file>       # last lines, live-updating (great for logs)
```

less is your friend for large files – it doesn't load the whole thing into memory.

SEARCHING WITH `grep`

```
1 grep "ERROR" results.txt      # find matching lines
2 grep -r -i "error" logs/      # recursive, case-insensitive
3 grep -n -l "TODO" *.py        # show line numbers / list matching files
```

- `-r` — recurse into directories
- `-i` — case-insensitive
- `-n` — prefix output with line numbers
- `-l` — list only filenames that match

USERS AND PERMISSIONS

READING PERMISSIONS

Linux is multi-user: every file has an **owner**, a **group**, and permissions for three audiences.

```
1 drwxr-xr-x  js056352  hpc-group  4096  testdir/  
2 -rw-r--r--  js056352  hpc-group    85  script.sh
```

```
[type] [user] [group] [other]  
d      rw-   r-x    r-x    ← dir: owner can write; others can read  
-      rw-   r--    r--    ← file: owner can write; others read-only
```

PERMISSION BITS

Bit	On a file	On a directory
r	Read	List contents (ls)
w	Modify	Create / delete inside
x	Execute	Enter (cd into)

ls -l shows permissions for: **[type][owner][group]**
[others]

CHANGING PERMISSIONS

```
1 chmod u+x script.sh      # make executable for owner
2 chmod go-w file.txt      # remove write for group and others
3 chmod a+r file.txt       # make readable for everyone
4
5 chown newuser file        # change owner (requires root)
6 chown user:group file     # change owner and group
```

Format: [u/g/o/a] [+/-] [r/w/x]

On HPC: "permission denied" and quota exceeded are the two most common filesystem errors.

THE VIM TEXT EDITOR

THREE MODES – THIS IS THE KEY INSIGHT

vim is always available on any Linux system.

When lost: press **Esc** repeatedly, then **u** to undo.

```

      Normal Mode      ← start here, always return here with Esc
      /               \
press :               press i / a / o
  ↓
  ↓
Command Mode      Insert Mode
(:w :q :wq :q!)   (type text normally)
```

OPENING, EDITING, SAVING

```
1 vim script.sh      " open a file
2 i                  " enter Insert mode – type normally
3 Esc                " return to Normal mode
4
5 :w                 " save
6 :wq                " save and quit
7 :q!                " quit WITHOUT saving
```

NORMAL MODE SHORTCUTS

Key	Action
dd / yy / p	Cut / copy / paste line
u / Ctrl-r	Undo / Redo
/pattern → n	Search forward
:%s/old/new/g	Replace all in file

*Esc always returns you to Normal mode
— when lost, press Esc first.*

ALTERNATIVES

- **nano** — keybindings shown at the bottom; good default for beginners
- **VS Code** with Remote SSH extension — full IDE experience on the cluster
- **MobaXTerm** (Windows) — has a built-in file editor

Use nano for quick config edits. Learn vim properly only if you spend significant time in the terminal.

SHELL SCRIPTS

ANATOMY OF A SHELL SCRIPT

A shell script is a text file of commands — the simplest form of automation.

```
1 #!/bin/bash
2 # Shebang — must be first line, tells Linux which interpreter
3 # #SBATCH directives (seen later) also use this comment syntax
4
5 echo "Starting..." # print to stdout — useful for logging
6 ls -l
7
8 chmod u+x myscript.sh # make executable
9 ./myscript.sh         # run — ./ needed because shell only
```

VARIABLES

```
1 name="world"           # assign — no spaces around =
2 echo "Hello $name"    # use with $
3
4 result=$(hostname)    # $() captures command output into a variable
5 # Quoting matters:
6 echo '$name'          # literal:    $name
7 echo "$name"          # expands:    world
```


SCRIPT ARGUMENTS

Passed positionally — \$1, \$2, ... (\$0 = script name, \$@ = all args):

```
# ./submit.sh 8 24:00:00 → cores=8, walltime=24:00:00  
cores=$1  
walltime=$2  
sbatch --cpus-per-task=$cores --time=$walltime myjob.sh
```

LOOPS AND CONDITIONALS

```
1 for f in *.txt; do
2     echo "Processing $f"
3 done
4
5 if [ -e "$filename" ]; then
6     echo "$filename exists."
7 fi
```

- Shell scripts are great for pipelines and automation glue
- For complex logic, data structures, or debugging — use **Python** instead

*Always write `#!/bin/bash` explicitly
— never assume the user's shell.*

ENVIRONMENT VARIABLES

ENVIRONMENT VARIABLES

A set of named variables available to the shell and all child processes. Convention: ALL_CAPS.

```
1 env
2           # list all environment variables
3 echo $HOME
4           # your home directory
5 echo $USER
6           # your username
7 export MY_VAR="value"           # create + export to child process
```

THE PATH VARIABLE

PATH — directories the shell searches for commands, : separated, first match wins:

```
1 echo $PATH
2 export PATH="$HOME/bin:$PATH" # prepend your own scripts dir
```

Similar variables: PYTHONPATH, LD_LIBRARY_PATH,
CUDA_HOME, ...

ENVIRONMENT MODULES

On HPC clusters, different users need different software versions → **environment modules**

```
module avail
    # list available modules
module load openmpi/gcc/64/1.10.3    # load a module
module list
    # show currently loaded modules
module unload openmpi/gcc/64/1.10.3 # unload when done
```

- Each module prepends to PATH and sets related variables
- Loading a module **never affects other users**
- Always load modules in your **job script** — never rely on interactive session state

SYSTEM CONFIGURATION

PERSONALIZING YOUR SHELL

Settings made in the terminal are **temporary** — they disappear on logout. To make them permanent, add them to `~/ .bashrc`:

```
1 # Aliases – shortcuts for long commands
2 alias ll='ls -l'
3 alias grep='grep --color=auto'
4
5 # Custom PATH
6 export PATH="$HOME/bin:$PATH"
7
8 source ~/.bashrc # apply changes without logging out
```

Other config files: `~/ .bash_profile` (login shells) •
`~/ .vimrc` (vim) • `~/ .ssh/config` (SSH)

LOCALES

Linux uses **locales** for language, encoding, and date/number formatting. Mismatched locales can cause subtle bugs (wrong sort order, garbled text output).

```
1 locale          # show current settings
2 echo $LANG      # e.g. de_DE.UTF-8 or en_US.UTF-8
```

If a program behaves unexpectedly with text — check your locale:

```
export LANG=en_US.UTF-8
```

SLURM JOB MANAGER

Simple Linux Utility for Resource Management

- Job scheduler often used in supercomputers and compute clusters
- Provides many advantages for utilizing HPC hardware with many users, such as..
 - ... Accounting, Containerization, Priorities, Chain- and Array-Jobs, ...

SLURM JOB MANAGER

- Users can interact with SLURM from login nodes
- Users may request cores, memory and time, then send their programs to be queued
- SLURM reserves these resources and waits till they are available
- Once available, the code will then be run

SLURM JOB MANAGER

SLURM JOB MANAGER

SLURM is fed **jobscrip**ts, which contain all information the scheduler needs to run a program

These jobscrip

ts consist of three parts:

1. Shebang
2. Job Parameters
3. Actual Job Code

SLURM JOB MANAGER

EXAMPLE

SLURM JOB MANAGER

EXAMPLE

```
#!/usr/bin/zsh
```

SLURM JOB MANAGER

EXAMPLE

```
#!/usr/bin/zsh

### Job Parameters
#SBATCH --cpus-per-task=8

#SBATCH --time=00:15:00
#SBATCH --job-name=example_job
#SBATCH --output=stdout.txt
#SBATCH --account=<project-id>
```


SLURM JOB MANAGER

EXAMPLE

```
#!/usr/bin/zsh

### Job Parameters
#SBATCH --cpus-per-task=8

#SBATCH --time=00:15:00
#SBATCH --job-name=example_job
#SBATCH --output=stdout.txt
#SBATCH --account=<project-id>

### Program Code
echo "Hello SLURM"
```

SLURM JOB MANAGER

EXAMPLE

- Save the file
- Submit the job

```
> sbatch testjob.sh
```

- Check its state

```
> squeue --me
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES
12345678	c23ms	example_job	AB123456	R	0:02	1

SLURM JOB MANAGER

COMMON PARAMETERS

- Number of cores: `-c /--cpus-per-task <numpcus>`
- Memory: `-m /--mem=<amount>G`
- Human readable Job name: `-J /--job-name`
- Reporting File: `-o /--output=<filename>`
- Runtime: `-t /--time=d-hh:mm:ss`
- Account: `-A /--account=<project>`
- GPUs: `--gres=gpu:<type>:<amount>`

SLURM JOB MANAGER

ESSENTIAL COMMANDS

```
# Submit a job  
sbatch myjob.sh
```

```
# Check the queue  
squeue --me  
    # your jobs only  
squeue --me --start          # estimated start time
```

```
# Cancel jobs  
scancel <JOB-ID>  
scancel --me  
    # cancel all your jobs
```

SLURM JOB MANAGER

INTERACTIVE JOBS AND ACCOUNTING

```
# Request an interactive session (great for testing)
salloc --gres=gpu:1 -n 24 -t 1:00:00    # 24 cores + 1 GPU for 1h
# Shell redirects to the compute node; job ends when you exit

# Show past and running job details
sacct -S $(date -I --date="yesterday") # all jobs since yesterday

# RWTH-specific: check quota usage
r_wlm_usage
```


SLURM JOB MANAGER

COMPUTE TIME APPLICATION

- Resources are measured in core hours (core-h)
 - A Macbook Pro has roughly 70k core-h/year
- The smallest project already guarantees 360k core-h
- Estimate your needs based on previous work
- Write and submit your application!

THE HPC PERFORMANCE PYRAMID

HPC resources in Germany are arranged hierarchically:

- **Tier-0:** PRACE and EuroHPC (European scale)
- **Tier-1:** Gauss Center for Supercomputing (GCS, JSC, HLRS, LRZ)
- **Tier-2:** Supra-regional HPC centres — NHR (includes RWTH CLAIX)
- **Tier-3:** Regional HPC centres — RWTH local projects

WHICH PROJECT TYPE?

Project	Core-h / year	Tier	Who
RWTH Thesis	< 0.048 Mio	3	Bachelor / Master thesis
RWTH Lecture	< 0.048 Mio	3	Teaching & courses
RWTH Small	< 0.36 Mio	3	Any RWTH/UKA researcher
NHR Normal	1 – 8 Mio	2	NRW universities (+ 2 Mio GPU-h)

NHR PROJECT TYPES

	PREP	Normal	Large
Scale	Starter quota	up to 8 Mio Core-h/year	12 – 50 Mio Core-h/year
GPUs	—	max. 2 Mio GPU-h	max. 4 Mio GPU-h
Duration	max. 6 months	continuous, renewable	fixed term
Review	none	technical + scientific	competitive
Deadlines	rolling	rolling	OCT · JAN ·

NHR NORMAL – APPLICATION PROCESS

1. Submit application (< 8 Mio Core-h)
2. **Technical Review** (< 2 weeks) — issues? resubmit with fixes
3. **Start contingent** provisioned (0.24 Mio Core-h/year)
— you can start computing now
4. **Scientific Review** (< 5 weeks)
5. Conditional Allocation → **RAB Decision** → Final resource allocation

The start contingent lets you begin work before the scientific review completes.

ACKNOWLEDGEMENT (RWTH/JARA)

Computations were performed with computing resources granted by RWTH Aachen University under project .

ACKNOWLEDGEMENT (RWTH/JARA)

The authors gratefully acknowledge the computing time provided to them at the NHR Center NHR4CES at RWTH Aachen University (project number <your-project-id: p0020XXX>). This is funded by the Federal Ministry of Education and Research, and the state governments participating on the basis of the resolutions of the GWK for national high performance computing at universities (www.nhr-verein.de/unser-partner).

TIPS AND TRICKS

VS CODE ON THE CLUSTER

Install the **Remote - SSH** extension, then connect using your `~/ .ssh/config` alias:

```
Host claix
  HostName login18-1.hpc.itc.rwth-aachen.de
  User ab123456
```

- Full IDE — file browser, terminal, git, debugger
- Python, Jupyter, and other extensions work remotely
- Edit files directly on the cluster — no file transfers needed

*Connect to login nodes for editing;
submit jobs from the integrated terminal.*

SLURM ARRAY JOBS

Run the same script with different inputs — one submission, many jobs:

```
1 #SBATCH --array=1-100          # submit 100 jobs at once
2
3 input_file="data/sample_${SLURM_ARRAY_TASK_ID}.txt"
4 python train.py --input "$input_file" --seed "$SLURM_ARRAY_TASK_ID"
```

- `$SLURM_ARRAY_TASK_ID` gives each job a unique index (1–100 here)
- `--array=0–99%10` — run at most 10 at a time to avoid flooding the queue
- Great for: parameter sweeps, cross-validation folds, processing file lists

PERSISTENT SESSIONS WITH TMUX

SSH sessions die when you disconnect — tmux keeps processes running:

```
1 tmux new -s work          # create a named session
2 # ... run long commands, then disconnect safely ...
3 tmux detach               # Ctrl-b d - detach (session stays alive)
4 tmux attach -t work       # reattach from any login node
```

- Run interactive jobs or monitoring without fear of losing the session
- Split panes: `Ctrl-b %` (vertical) · `Ctrl-b "` (horizontal)
- List sessions: `tmux ls`

JOB DEPENDENCIES – CHAINING JOBS

Submit a pipeline without waiting for each step to finish:

```
1 jid1=$(sbatch --parsable preprocess.sh)
2 jid2=$(sbatch --dependency=afterok:$jid1 --parsable train.sh)
3 sbatch --dependency=afterok:$jid2 evaluate.sh
```

Condition	Meaning
afterok:ID	Start only if job ID succeeded
afterany:ID	Start after job ID ends (any status)
afternotok:ID	Start only if job ID failed

PYTHON ENVIRONMENTS ON THE CLUSTER

Don't install packages globally — use isolated environments:

```
1 module load Miniconda3          # load conda (check: module)
2 conda create -n myenv python=3.11 # create environment
3 conda activate myenv
4 # activate
5 pip install -r requirements.txt   # install packages
```

In your job script, always activate the environment explicitly:

```
module load Miniconda3
conda activate myenv
python train.py
```

ANY QUESTIONS?